

Cyber AI Platform MVP

Technical Design

MVP Scope: Vulnerability Chaining, Zero Blast Radius, NL2SQL, Threat Modeling, Risk Ranker, PKI Certificate Expiry

Generated for enterprise architecture discussion

Design Principle	Meaning
Orchestrator proposes	The orchestrator classifies intent and builds a plan, but it does not enforce security.
Workflow registry constrains	Only approved workflow definitions and versions can run.
Policy service authorizes	All workflow, agent, tool, data, and action decisions go through policy.
Workflow runtime tracks	Long-running state, progress, retries, approvals, and cancellation are durable.
Agent runtime executes	Approved agents run bounded tasks and call tools only through the gateway.
Tool gateway enforces	Tools are schema-checked, policy-gated, masked, rate-limited, and audited.
Verifier finalizes	Responses are grounded, redacted, confidence-scored, and evidence-linked.

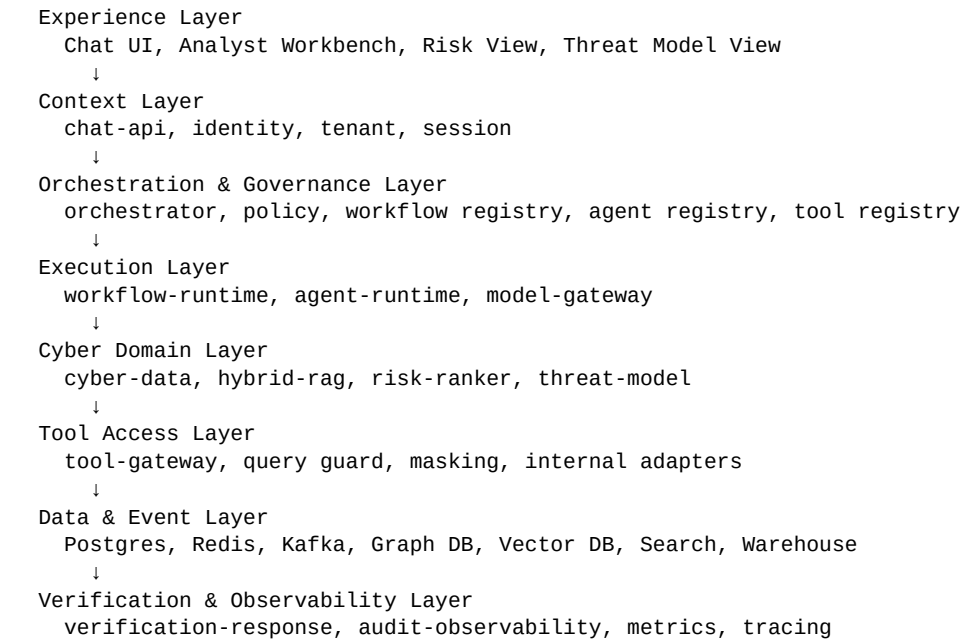
1. MVP Goals and Scope

The MVP builds a governed Cyber AI platform focused on six high-value cyber capabilities. It is read-mostly, with write actions limited to draft-only or approval-gated paths.

Capability	User Questions the MVP Should Answer
Vulnerability Chaining	Can these vulnerabilities combine into a high-impact path? Which chains matter most?
Zero Blast Radius	If this asset/app is compromised, what downstream systems and data are reachable?
NL2SQL	Show top critical vulnerabilities, expiring certs, exposed assets, and risk trends using natural language.
Threat Modeling	Generate a threat model for an application, API, or architecture using evidence and templates.
Risk Ranker	Rank risks across findings, apps, assets, certificates, and threat scenarios.
PKI Certificate Expiry	Which production or internet-facing certificates expire soon, and what is the impact?

- Out of MVP: autonomous patching, firewall changes, raw PII access, dedicated connector-service, third-party risk, full workflow builder UI.
- Connector capability is implemented inside the Tool Gateway for MVP as internal adapters.
- Every answer must include evidence, risk rationale, confidence, limitations, and recommended next action.

2. High-Level Architecture



Layer	Main Responsibility
Experience	User interaction: chat, workbench, evidence, risk,

	workflow progress.
Context	Identity, tenant boundary, session state, active case memory.
Orchestration & Governance	Intent, workflow selection, policy, registries, plan validation.
Execution	Durable workflow execution and approved agent runtime.
Cyber Domain	Data products, graph/RAG retrieval, threat models, risk ranking.
Tool Access	Tool authorization, query guard, masking, internal adapters.
Data & Event	Operational DB, cache, event bus, graph, vector, search, warehouse.
Verification & Observability	Grounding, redaction, response building, audit, traces, metrics.

3. MVP Microservices

#	Microservice	Description
1	chat-api-service	User-facing API for chat, streaming, workbench, workflow status, and evidence views.
2	identity-service	Enterprise identity enrichment: user profile, roles, groups, MFA, clearance, status.
3	tenant-service	Resolves tenant, data boundary, environment scope, and tenant-level policy bindings.
4	session-service	Stores conversation turns, active case, extracted entities, and workflow links.
5	orchestrator-service	Classifies intent, selects workflow, generates plan, decides sync/async execution.
6	policy-service	Runtime allow/deny/mask/limit/approval decisions.
7	workflow-registry-service	Approved workflow definitions, versions, validation, and published contracts.
8	agent-registry-service	Approved agents, versions, owners, input/output schemas, allowed tools.
9	tool-registry-service	Approved tools, schemas, read/write classification, limits, data classification.
10	workflow-runtime-service	Durable workflow lifecycle, steps, retries, progress, heartbeats,

		approvals.
11	agent-runtime-service	Executes approved agents: vuln chaining, blast radius, NL2SQL, PKI, evidence.
12	tool-gateway-service	Enforces tool access, query guard, masking, rate limits, and internal adapters.
13	hybrid-rag-service	Vector, graph, keyword, SQL/data product, log/runbook retrieval and evidence ranking.
14	cyber-data-service	Governed cyber data products, graph paths, semantic models, evidence store.
15	risk-ranker-service	Scores and ranks cyber risks across findings, apps, assets, certificates, threats.
16	threat-model-service	Generates evidence-backed threat models with STRIDE-style analysis.
17	verification-response-service	Grounding, redaction, confidence, risk explanation validation, response builder.
18	model-gateway-service	Central LLM and embedding access, prompt templates, token/cost limits.
19	audit-observability-service	Audit events, traces, metrics, cost, latency, execution history.

Recommended MVP physical deployment can reduce operational overhead while preserving logical boundaries:

1. chat-api-service
2. context-api
 - identity router
 - tenant router
 - session router
3. orchestrator-service
4. governance-api
 - policy router
 - workflow registry router
 - agent registry router
 - tool registry router
5. workflow-runtime-service
6. agent-runtime-service
7. tool-gateway-service
8. cyber-domain-api
 - hybrid RAG router
 - cyber data router
 - risk ranker router
 - threat model router
9. verification-response-service
10. model-gateway-service
11. audit-observability-service

4. Service Components and APIs

chat-api-service

Responsibilities:

- Builds request context
- Streams answers
- Shows workflow status and evidence

Key endpoints:

```
POST /v1/chat/messages
GET /v1/workflows/{execution_id}/status
GET /v1/evidence/{evidence_id}
```

identity-service

Responsibilities:

- Enriches enterprise identity
- Resolves roles/groups
- Used at request start and high-risk approvals

Key endpoints:

```
GET /v1/identity/me
POST /v1/identity/resolve
```

tenant-service

Responsibilities:

- Resolves tenant
- Returns allowed data products
- Returns tenant policy bindings

Key endpoints:

```
POST /v1/tenants/resolve
GET /v1/tenants/{tenant_id}/data-boundary
```

session-service

Responsibilities:

- Stores turns and entities
- Tracks active case
- Links execution_id to session

Key endpoints:

```
POST /v1/sessions
GET /v1/sessions/{id}/context
POST /v1/sessions/{id}/link-workflow
```

orchestrator-service

Responsibilities:

- Classifies intent
- Selects workflow
- Generates and validates plan

Key endpoints:

```
POST /v1/orchestrate
POST /v1/intents/classify
POST /v1/plans/validate
```

policy-service

Responsibilities:

- Returns ALLOW, DENY, MASK, LIMIT, APPROVAL_REQUIRED, STEP_UP_AUTH_REQUIRED

Key endpoints:

```
POST /v1/policy/evaluate
```

workflow-runtime-service

Responsibilities:

- Owns lifecycle
- Tracks steps, progress, retries
- Publishes Kafka events

Key endpoints:

```
POST /v1/workflows/run
GET /v1/executions/{id}
POST /v1/executions/{id}/heartbeat
```

agent-runtime-service

Responsibilities:

- Executes approved agents
- Calls model gateway and tool gateway
- Returns structured evidence-backed output

Key endpoints:

```
POST /v1/agents/execute
POST /v1/agents/nl2sql/generate
POST /v1/agents/pki/expiry-analysis
```

tool-gateway-service

Responsibilities:

- Policy-gates every tool call
- Validates SQL/graph/search

- Contains MVP adapters

Key endpoints:

```
POST /v1/tools/invoke
POST /v1/queries/validate
POST /v1/mask
```

hybrid-rag-service

Responsibilities:

- Retrieves vector/graph/search/SQL evidence
- Ranks evidence by relevance/freshness

Key endpoints:

```
POST /v1/rag/retrieve
POST /v1/evidence/rank
```

cyber-data-service

Responsibilities:

- Serves data products
- Provides graph paths
- Stores evidence bundles

Key endpoints:

```
GET /v1/data-products/vulnerabilities
POST /v1/graph/paths
POST /v1/evidence/store
```

risk-ranker-service

Responsibilities:

- Scores exploitability, exposure, criticality, data sensitivity, blast radius, controls

Key endpoints:

```
POST /v1/risk/rank
POST /v1/risk/score
POST /v1/risk/explain
```

threat-model-service

Responsibilities:

- Builds app context
- Uses STRIDE methodology
- Produces mitigations and residual risk

Key endpoints:

```
POST /v1/threat-models/generate
GET /v1/threat-models/{id}
```

verification-response-service

Responsibilities:

- Checks grounding
- Applies redaction
- Builds final answer

Key endpoints:

POST /v1/verify
 POST /v1/responses/build

5. Cyber Data Products

Data Product	Purpose
asset_inventory	Assets, hosts, environments, criticality, owner, internet-facing flag.
application_ownership	Applications, owners, business unit, customer-facing flag, data domains.
vulnerability_findings	Normalized scanner findings, CVE, severity, CVSS, EPSS, KEV, status.
network_exposure	Reachability, zones, internet exposure, upstream/downstream relationships.
data_classification_summary	Dataset classification, customer data flag, summary-only restrictions.
certificate_inventory	Certificates, expiry, CN/SAN, app mapping, internet-facing, auto-renewal.
risk_findings	Ranked risk entities, scores, rationales, SLA recommendations.
threat_model_evidence	Architecture docs, data flows, trust boundaries, threat scenarios.
evidence_bundles	Evidence references, graph paths, query results, source freshness.

6. Workflow Definitions

Workflow	Intent	Core Steps
vulnerability_chaining_workflow	vulnerability_chaining	Find findings -> graph paths -> data impact -> chain risk -> recommendations
zero_blast_radius_workflow	zero_blast_radius	Input asset/app -> graph neighborhood -> downstream systems -> data impact -> blast radius score
nl2sql_cyber_query_workflow	cyber_nl2sql_query	Semantic model retrieval -> SQL generation -> query guard -> execution -> result verification
threat_model_generation_workflo	threat_model_generation	App context -> architecture docs

w		-> STRIDE model -> mitigations -> residual risk
risk_ranking_workflow	risk_ranking	Collect entities -> normalize features -> score -> rank -> explain
pki_certificate_expiry_workflow	pki_certificate_expiry	Certificate inventory -> app mapping -> expiry risk -> owner actions

Example workflow contract fields:

workflow_id: pki_certificate_expiry_workflow

version: 1.0.0

owner: pki-security

allowed_agents: [pki_expiry_agent, risk_explanation_agent]

allowed_tools: [certificate_inventory_tool, cmdb_lookup_tool, graph_query_tool, runbook_retrieval_tool, jira_draft_tool]

blocked_actions: [revoke_certificate, rotate_certificate_without_approval, production_change_without_approval]

approval_required_for: [create_jira, notify_app_owner, rotate_certificate]

output_contract: { evidence_required: true, pii_mode: masked }

7. Redis Design

Redis is used for short-lived state, locks, counters, and performance optimization. It is not the durable source of truth.

Redis Use Case	Key Pattern	TTL	Notes
Active session cache	session: {session_id}:context	30 min	Active case, last entities, tenant/user summary.
Idempotency	idem:{request_id}	24 h	Prevents duplicate workflow starts.
Workflow lock	lock:workflow: {execution_id}	60 sec	Worker lease for state transitions.
Workflow progress	workflow: {execution_id}:progress	6 h	Fast UI polling/SSE state.
Agent heartbeat	agent: {agent_execution_id}:heartbeat	5 min	Detect stale agent task.
Tool rate limit	ratelimit:tool: {tool_id}:user:{user_id}	60 sec	Counters for throttling.
Model budget	budget:model:user: {user_id}:day:{date}	24 h	Token and cost governance.
RAG cache	rag:{tenant_id}:{hash}	10 min	Ranked evidence refs for repeated questions.

- Do not store final workflow state only in Redis.
- Do not store long-term audit only in Redis.
- Postgres and object storage remain durable stores of record.

8. Kafka Design

Kafka is used for asynchronous workflow execution, agent commands, telemetry, audit streaming, and decoupling services.

Topic	Purpose	Partition Key
cyber.workflow.commands	Start, resume, cancel workflow commands.	execution_id
cyber.workflow.events	WorkflowStarted, StepStarted, StepCompleted, ApprovalRequired, Completed, Failed.	execution_id
cyber.agent.commands	Dispatch long-running agent tasks.	agent_execution_id
cyber.agent.events	Agent results, heartbeats, failures.	agent_execution_id
cyber.tool.events	Tool-call telemetry and summaries.	tool_call_id
cyber.policy.events	Policy decision stream.	execution_id or request_id
cyber.audit.events	Append-only audit event stream.	tenant_id or execution_id
cyber.evidence.events	Evidence created/updated.	evidence_id
cyber.notification.events	Workflow completion, approval, UI notifications.	user_id or execution_id
cyber.dlq	Dead-letter events for replay/investigation.	source event key

Standard Kafka event envelope:

```
{
  "event_id": "evt-001",
  "event_type": "StepCompleted",
  "event_version": "1.0",
  "timestamp": "2026-06-01T10:00:00Z",
  "tenant_id": "cyber-prod",
  "user_id": "u123",
  "correlation_id": "req-123",
  "execution_id": "exec-456",
  "payload": {}
}
```

9. Postgres Operational Schema

Table	Purpose
sessions	Chat sessions and metadata.
conversation_turns	User and assistant messages, tool/reference links.
workflow_definitions	Workflow metadata.
workflow_versions	Workflow YAML/JSON versions and validation state.
agent_registry	Agent metadata and versions.
tool_registry	Tool metadata and versions.
workflow_executions	Durable workflow execution state.
workflow_steps	Per-step status and output refs.

agent_executions	Agent status, heartbeat, progress, errors.
tool_calls	Tool-call summaries, evidence refs, errors.
policy_decisions	Authorization/masking/approval decisions.
approval_requests	Human approval state for gated actions.
evidence_refs	Evidence metadata, source refs, classification, freshness.
audit_events	Durable audit event records.

```
CREATE TABLE workflow_executions (
  execution_id UUID PRIMARY KEY,
  workflow_id TEXT NOT NULL,
  workflow_version TEXT NOT NULL,
  tenant_id TEXT NOT NULL,
  user_id TEXT NOT NULL,
  session_id UUID,
  status TEXT NOT NULL,
  input_json JSONB NOT NULL,
  output_json JSONB,
  current_step TEXT,
  progress_percent INT DEFAULT 0,
  created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  started_at TIMESTAMPTZ,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  completed_at TIMESTAMPTZ,
  error_json JSONB
);
```

10. Key Sequence Flows

Vulnerability Chaining

User -> chat-api -> orchestrator -> workflow-registry -> policy -> workflow-runtime -> agent-runtime -> tool-gateway -> cyber-data/hybrid-rag -> risk-ranker -> verification-response -> user

Zero Blast Radius

User asks about asset/app -> graph_query_tool -> cyber-data graph paths -> risk-ranker scores blast radius -> verifier redacts and builds response.

NL2SQL

User question -> NL2SQL agent -> semantic model retrieval -> SQL generation -> query guard -> policy table/column check -> governed data product execution -> verification.

Threat Modeling

User selects app -> threat-model-service retrieves architecture, graph, data classification -> model-gateway generates STRIDE draft -> verifier grounds output.

PKI Expiry

User asks expiring certs -> PKI expiry agent -> certificate_inventory_tool -> cyber-data certificate product -> risk-ranker -> verifier -> response.

11. Tool Gateway and Query Guard

- Agents never call enterprise systems directly.
- Connector-service is deferred; internal adapters live inside Tool Gateway for MVP.

- Every tool call validates tool registry metadata, policy, schema, tenant filter, limits, and masking.

MVP Tool	Purpose
vulnerability_findings_tool	Retrieve normalized vulnerability findings.
asset_inventory_tool	Retrieve asset metadata, criticality, owner, environment.
application_ownership_tool	Map assets/services to applications and owners.
network_exposure_tool	Check internet exposure and network zones.
graph_query_tool	Find relationship paths across app/asset/data graph.
data_classification_tool	Retrieve data sensitivity summaries.
certificate_inventory_tool	Retrieve certificate expiry and app mapping.
semantic_model_tool	Retrieve semantic model for NL2SQL.
sql_query_tool	Execute validated SQL against governed views.
runbook_retrieval_tool	Retrieve runbooks and operating procedures.
architecture_doc_retrieval_tool	Retrieve architecture docs for threat modeling.
jira_draft_tool	Generate Jira draft; creation is approval-gated.

NL2SQL Query Guard checks:

- No SELECT *
- Tenant filter required
- LIMIT required
- Only approved views/tables
- No restricted columns
- No DDL/DML
- No cross-tenant joins
- No raw PII access
- Query timeout and cost limit enforced

12. Long-Running Workflow Tracking

State Type	Values
Workflow	CREATED, QUEUED, RUNNING, WAITING_ON_TOOL, PAUSED_FOR_APPROVAL, WAITING_ON_USER_INPUT, RETRYING, COMPLETED, FAILED, CANCELLED, TIMED_OUT, PARTIALLY_COMPLETED
Step	PENDING, RUNNING, COMPLETED, FAILED, SKIPPED, RETRYING, TIMED_OUT
Agent	CREATED, RUNNING, HEARTBEATING, COMPLETED, FAILED, STALE, TIMED_OUT

Workflow Runtime owns:

- execution_id
- current step
- progress percent
- retries and timeouts
- approval pauses
- cancellation
- final status

Agent Runtime owns:

- agent_execution_id
- agent heartbeat

- agent progress
- structured result

Session Service links:

- session_id -> active case -> execution_id

13. Security and Governance

Control	MVP Implementation
Authentication	Shared FastAPI auth library with JWT/JWKS; service-to-service JWT or mTLS.
Authorization	Policy service returns ALLOW/DENY/MASK/LIMIT/APPROVAL_REQUIRED.
Tenant isolation	Tenant-service resolves data boundary; tool gateway enforces tenant filters.
Data protection	Column restrictions, row limits, restricted summary-only mode, masking.
Secrets	Vault/cloud secrets; no secrets in prompts or agent memory.
Audit	Every request, policy decision, workflow step, agent execution, and tool call logged.
Output safety	Verification-response service grounds, redacts, confidence-scores all answers.

14. Deployment and Technology Stack

Capability	MVP Technology
API services	FastAPI, Uvicorn/Gunicorn, Python 3.11+
Operational store	Postgres
Cache/locks/rate limits	Redis
Async events	Kafka
Search	OpenSearch
Vector retrieval	pgvector or OpenSearch vector
Graph	Neo4j, PuppyGraph, or Spanner Graph
Warehouse	Snowflake, BigQuery, or Databricks
Object storage	S3, GCS, or ADLS for evidence bundles
Secrets	Vault or cloud secret manager
Observability	OpenTelemetry, Prometheus, Grafana, Splunk/OpenSearch logs
Deployment	Kubernetes / OpenShift

Recommended workers:

- workflow-runtime-worker
- agent-runtime-worker
- audit-event-consumer
- tool-event-consumer
- rag-indexing-worker
- workflow-notification-worker

15. MVP Build Phases

Phase	Services	Outcome
Phase 1 - Foundation	chat, context, orchestrator, policy, workflow registry, model gateway, audit	Authenticated chat, tenant/session context, workflow selection, policy, audit.
Phase 2 - Execution	workflow runtime, agent runtime, agent registry, tool registry, tool gateway	Durable workflows, agents, approved tools, safe tool calls.
Phase 3 - Domain	cyber data, hybrid RAG, risk ranker, threat model	Six MVP capabilities with evidence retrieval and scoring.
Phase 4 - Hardening	verification, query guard, redaction, Kafka consumers, Redis controls, dashboards	Production-grade safety, observability, and response quality.

16. Final Architecture Summary

The MVP is a policy-gated hybrid workflow orchestration platform for six cyber AI capabilities. It uses FastAPI microservices, Postgres for durable state, Redis for cache/locks/progress/rate limits, Kafka for async workflow and audit events, Tool Gateway for safe enterprise data access, Hybrid RAG for evidence retrieval, and Verification Response for grounded, redacted outputs.